JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class RuleBasedCollator

java.lang.Object
    java.text.Collator
        java.text.RuleBasedCollator

**All Implemented Interfaces:**

`Cloneable, Comparator<Object>`

---

```
public class RuleBasedCollator
extends Collator
```

The `RuleBasedCollator` class is a concrete subclass of `Collator` that provides a simple, data-driven, table collator. With this class you can create a customized table-based `Collator`. `RuleBasedCollator` maps characters to sort keys.

`RuleBasedCollator` has the following restrictions for efficiency (other subclasses may be used for more complex languages) :

1. If a special collation rule controlled by a <modifier> is specified it applies to the whole collator object.
2. All non-mentioned characters are at the end of the collation order.

The collation table is composed of a list of collation rules, where each rule is of one of three forms:

```
<modifier>
<relation> <text-argument>
<reset> <text-argument>
```

The definitions of the rule elements is as follows:

- **Text-Argument**: A text-argument is any sequence of characters, excluding special characters (that is, common whitespace characters [0009-000D, 0020] and rule syntax characters [0021-002F, 003A-0040, 005B-0060, 007B-007E]). If those characters are desired, you can put them in single quotes (e.g. ampersand => '&'). Note that unquoted white space characters are ignored; e.g. `b c` is treated as `bc`.
- **Modifier**: There are currently two modifiers that turn on special collation rules.
    - '@' : Turns on backwards sorting of accents (secondary differences), as in French.
    - '!' : Turns on Thai/Lao vowel-consonant swapping. If this rule is in force when a Thai vowel of the range \U0E40-\U0E44 precedes a Thai consonant of the range \U0E01-\U0E2E OR a Lao vowel of the range \U0EC0-\U0EC4 precedes a Lao consonant of the range

\U0E81-\U0EAE then the vowel is placed after the consonant for collation purposes.

'@' : Indicates that accents are sorted backwards, as in French.

- **Relation**: The relations are the following:
  - '<' : Greater, as a letter difference (primary)
  - ';' : Greater, as an accent difference (secondary)
  - ',' : Greater, as a case difference (tertiary)
  - '=' : Equal
- **Reset**: There is a single reset which is used primarily for contractions and expansions, but which can also be used to add a modification at the end of a set of rules.

  '&' : Indicates that the next rule follows the position to where the reset text-argument would be sorted.

This sounds more complicated than it is in practice. For example, the following are equivalent ways of expressing the same thing:

```
a < b < c
a < b & b < c
a < c & a < b
```

Notice that the order is important, as the subsequent item goes immediately after the text-argument. The following are not equivalent:

```
a < b & a < c
a < c & a < b
```

Either the text-argument must already be present in the sequence, or some initial substring of the text-argument must be present. (e.g. "a < b & ae < e" is valid since "a" is present in the sequence before "ae" is reset). In this latter case, "ae" is not entered and treated as a single character; instead, "e" is sorted as if it were expanded to two characters: "a" followed by an "e". This difference appears in natural languages: in traditional Spanish "ch" is treated as though it contracts to a single character (expressed as "c < ch < d"), while in traditional German a-umlaut is treated as though it expanded to two characters (expressed as "a,A < b,B ... &ae;\u00e3&AE;\u00c3"). [\u00e3 and \u00c3 are, of course, the escape sequences for a-umlaut.]

**Ignorable Characters**

For ignorable characters, the first rule must start with a relation (the examples we have used above are really fragments; "a < b" really should be "< a < b"). If, however, the first relation is not "<", then all the all text-arguments up to

the first "<" are ignorable. For example, ", - < a < b" makes "-" an ignorable character, as we saw earlier in the word "black-birds". In the samples for different languages, you see that most accents are ignorable.

**Normalization and Accents**

`RuleBasedCollator` automatically processes its rule table to include both pre-composed and combining-character versions of accented characters. Even if the provided rule string contains only base characters and separate combining accent characters, the pre-composed accented characters matching all canonical combinations of characters from the rule string will be entered in the table.

This allows you to use a RuleBasedCollator to compare accented strings even when the collator is set to NO_DECOMPOSITION. There are two caveats, however. First, if the strings to be collated contain combining sequences that may not be in canonical order, you should set the collator to CANONICAL_DECOMPOSITION or FULL_DECOMPOSITION to enable sorting of combining sequences. Second, if the strings contain characters with compatibility decompositions (such as full-width and half-width forms), you must use FULL_DECOMPOSITION, since the rule tables only include canonical mappings.

**Errors**

The following are errors:

- A text-argument contains unquoted punctuation symbols (e.g. "a < b-c < d").
- A relation or reset character not followed by a text-argument (e.g. "a < ,b").
- A reset where the text-argument (or an initial substring of the text-argument) is not already in the sequence. (e.g. "a < b & e < f")

If you produce one of these errors, a `RuleBasedCollator` throws a `ParseException`.

**Examples**

Simple: "< a < b < c < d"

Norwegian: "< a, A < b, B < c, C < d, D < e, E < f, F < g, G < h, H < i, I < j, J < k, K < l, L < m, M < n, N < o, O < p, P < q, Q < r, R < s, S < t, T < u, U < v, V < w, W < x, X < y, Y < z, Z < \u00E6, \u00C6 < \u00F8, \u00D8 < \u00E5 = a\u030A, \u00C5 = A\u030A; aa, AA"

To create a `RuleBasedCollator` object with specialized rules tailored to your needs, you construct the `RuleBasedCollator` with the rules contained in a `String` object. For example:

```
String simple = "< a< b< c< d";
RuleBasedCollator mySimple = new RuleBasedCollator(simple);
```

Or:

```
String Norwegian = "< a, A < b, B < c, C < d, D < e, E < f, F < g, G < h, H <
                   "< j, J < k, K < l, L < m, M < n, N < o, O < p, P < q, Q <
                   "< s, S < t, T < u, U < v, V < w, W < x, X < y, Y < z, Z"
                   "< \u00E6, \u00C6" +     // Latin letter ae & AE
                   "< \u00F8, \u00D8" +     // Latin letter o & O with stroke
                   "< \u00E5 = a\u030A," +  // Latin letter a with ring above
                   "  \u00C5 = A\u030A;" +  // Latin letter A with ring above
                   "  aa, AA";
RuleBasedCollator myNorwegian = new RuleBasedCollator(Norwegian);
```

A new collation rules string can be created by concatenating rules strings. For example, the rules returned by `getRules()` could be concatenated to combine multiple `RuleBasedCollators`.

The following example demonstrates how to change the order of non-spacing accents,

```
// old rule
String oldRules = "=\u0301;\u0300;\u0302;\u0308"    // main accents
                + ";\u0327;\u0303;\u0304;\u0305"    // main accents
                + ";\u0306;\u0307;\u0309;\u030A"    // main accents
                + ";\u030B;\u030C;\u030D;\u030E"    // main accents
                + ";\u030F;\u0310;\u0311;\u0312"    // main accents
                + "< a , A ; ae, AE ; \u00e6 , \u00c6"
                + "< b , B < c, C < e, E & C < d, D";
// change the order of accent characters
String addOn = "& \u0300 ; \u0308 ; \u0302";
RuleBasedCollator myCollator = new RuleBasedCollator(oldRules + addOn);
```

**Implementation Note:**

For this implementation, concurrent usage of this class may lead to significant thread contention since `synchronized` is employed to ensure thread-safety. As such, users of this class should consider creating a separate instance for each thread when used in multithreaded environments.

**Since:**

1.1

**See Also:**

Collator, CollationElementIterator

## Field Summary

**Fields declared in class Collator**

CANONICAL_DECOMPOSITION, FULL_DECOMPOSITION, IDENTICAL, NO_DECOMPOSITION, PRIMARY, SECONDARY, TERTIARY

## Constructor Summary

**Constructors**

| Constructor | Description |
|---|---|
| RuleBasedCollator(String rules) | RuleBasedCollator constructor. |

## Method Summary

**All Methods** | **Instance Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| Object | clone() | Standard override; no change in semantics. |
| int | compare(String source, String target) | Compares the character data stored in two different strings based on the collation rules. |
| boolean | equals(Object obj) | Compares the equality of two collation objects. |
| CollationElementIter | getCollationElementItera (String source) | Returns a CollationElementIterator for the given String. |
| CollationElementIter | getCollationElementItera (CharacterIterator sourc | Returns a CollationElementIterator for the given CharacterIterator. |

| | | |
|---|---|---|
| CollationKey | **getCollationKey**(String source) | Transforms the string into a series of characters that can be compared with CollationKey.compareTo. |
| String | **getRules**() | Gets the table-based rules for the collation object. |
| int | **hashCode**() | Generates the hash code for the table-based collation object |

**Methods declared in class Collator**

compare, equals, getAvailableLocales, getDecomposition, getInstance, getInstance, getStrength, setDecomposition, setStrength

**Methods declared in class Object**

finalize, getClass, notify, notifyAll, toString, wait, wait, wait

**Methods declared in interface Comparator**

max, min, reversed, thenComparing, thenComparing, thenComparing, thenComparingDouble, thenComparingInt, thenComparingLong

## *Constructor Details*

### RuleBasedCollator

```
public RuleBasedCollator(String rules)
                  throws ParseException
```

RuleBasedCollator constructor. This takes the table rules and builds a collation table out of them. Please see RuleBasedCollator class description for more details on the collation rule syntax.

**Parameters:**

`rules` - the collation rules to build the collation table from.

**Throws:**

`ParseException` - A format exception will be thrown if the build process of the rules fails. For example, build rule "a < ? < d" will cause the constructor to throw the ParseException because the '?' is not quoted.

**See Also:**

`Locale`

## *Method Details*

### getRules

```
public String getRules()
```

Gets the table-based rules for the collation object.

**Returns:**

returns the collation rules that the table collation object was created from.

### getCollationElementIterator

```
public CollationElementIterator getCollationElementIterator
(String source)
```

Returns a CollationElementIterator for the given String.

**Parameters:**

`source` - the string to be collated

**Returns:**

a `CollationElementIterator` object

**See Also:**

`CollationElementIterator`

### getCollationElementIterator

```
public CollationElementIterator getCollationElementIterator
(CharacterIterator source)
```

Returns a CollationElementIterator for the given CharacterIterator.

**Parameters:**

`source` - the character iterator to be collated

**Returns:**

a `CollationElementIterator` object

**Since:**

1.2

**See Also:**

CollationElementIterator

## compare

```
public int compare(String source,
                   String target)
```

Compares the character data stored in two different strings based on the collation rules. Returns information about whether a string is less than, greater than or equal to another string in a language. This can be overridden in a subclass.

**Specified by:**

compare in class Collator

**Parameters:**

source - the source string.

target - the target string.

**Returns:**

Returns an integer value. Value is less than zero if source is less than target, value is zero if source and target are equal, value is greater than zero if source is greater than target.

**Throws:**

NullPointerException - if source or target is null.

**See Also:**

CollationKey,
Collator.getCollationKey(String)

## getCollationKey

```
public CollationKey getCollationKey(String source)
```

Transforms the string into a series of characters that can be compared with CollationKey.compareTo. This overrides java.text.Collator.getCollationKey. It can be overridden in a subclass.

**Specified by:**

getCollationKey in class Collator

**Parameters:**

source - the string to be transformed into a collation key.

**Returns:**

the CollationKey for the given String based on this Collator's collation rules. If the source String is null, a null CollationKey is returned.

**See Also:**

`CollationKey`,
`Collator.compare(String, String)`

## clone

```
public Object clone()
```

Standard override; no change in semantics.

**Overrides:**

`clone` in class `Collator`

**Returns:**

a clone of this instance.

**See Also:**

`Cloneable`

## equals

```
public boolean equals(Object obj)
```

Compares the equality of two collation objects.

**Specified by:**

`equals` in interface `Comparator<Object>`

**Overrides:**

`equals` in class `Collator`

**Parameters:**

`obj` - the table-based collation object to be compared with this.

**Returns:**

true if the current table-based collation object is the same as the table-based collation object obj; false otherwise.

**See Also:**

`Object.equals(Object)`, `Object.hashCode()`

## hashCode

```
public int hashCode()
```

Generates the hash code for the table-based collation object

**Specified by:**

`hashCode` in class `Collator`

**Returns:**

a hash code value for this object

**See Also:**

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.